FizzBuzz Project

Table of contents

README

User Documentation

What is this?

This program implements the classic FizzBuzz problem. The main logic is in `fizzbuzz.py`, and tests are in `test_fizzbuzz.py`. The function takes a single integer and returns:

- "Fizzbuzz" if the number is divisible by both 3 and 5
- "Fizz" if divisible by 3
- "Buzz" if divisible by 5
- The number as a string otherwise.

How do I use it?

1. Install dependencies if you do not already have them.
2. Use the guide for the FizzBuzz function

Installing Dependencies

- You need Python 3.x installed. To install Python, visit: https://www.python.org/downloads/
- If you want to run tests, you need to install `pytest`:

  ```
  pip install pytest
  ```

Using the fizzbuzz Function in Your Code

You can import the `fizzbuzz` function from `fizzbuzz.py` and use it in your own Python scripts or in the Python interactive shell. The function returns a string for a single number.

Example Usage

1. Open a Python shell or create a new Python script in the same directory.

2. Import the function:
   ```
   from fizzbuzz import fizzbuzz
   ```
3. Call the function with your desired range (e.g., 1 to 100):
   ```
   print(fizzbuzz(15))  # Output: Fizzbuzz
   print(fizzbuzz(9))   # Output: Fizz
   print(fizzbuzz(10))  # Output: Buzz
   print(fizzbuzz(7))   # Output: 7
   ```

This will print the FizzBuzz result for a single number.

Technical Documentation

- **fizzbuzz.py**: Contains the main logic for the FizzBuzz problem.
- **test_fizzbuzz.py**: Contains tests for the FizzBuzz implementation using pytest. (Note: The test file must be named with underscores, not dashes, for pytest to discover it.)
- **.github/workflows/fizzbuzz-tests.yml**: GitHub Actions workflow file that automatically runs the test suite on every push and pull request. It sets up a Python environment, installs dependencies, and runs pytest to ensure code quality and correctness.

The main function takes a single integer and returns the appropriate FizzBuzz string based on divisibility by 3 and/or 5.

Developer Documentation

Project Structure

- `fizzbuzz.py` : Main script.
- `test_fizzbuzz.py` : Test script (ensure the file is named with underscores, not dashes).
- `.github/workflows/fizzbuzz-tests.yml` : Continuous Integration (CI) workflow for automated testing with GitHub Actions.

Continuous Integration

All tests are automatically run on GitHub Actions for every push and pull request using the workflow defined in `.github/workflows/fizzbuzz-tests.yml` . This ensures that all code changes are tested before being merged.

How to Run Tests Locally

To run the tests locally, make sure you are in the project root directory. Run the following command:

```
pytest
```

Pytest will automatically discover and run all test files named `test_*.py` (with underscores).

Contribution Guidelines

- Follow PEP8 style guidelines.
- Write tests for new features or bug fixes.
- Document your code with comments where necessary.

Code for fizzbuzz function

Below you will find the code for the fizzbuzz function.

In [ ]:
```
def fizzbuzz(n):

    #negative value check

    if n <0:
        raise ValueError("Input must be a non negative number")


    if n % 15 == 0:
        return "Fizzbuzz"

    if n % 3 == 0:
        return "Fizz"
    if n % 5 == 0:
        return "Buzz"



    #return the input as strings
    return str(n)
```

Tests

Below is the code used to test the fizzbuzz function exists, the function behaves as expected and error handling works as expected.

In [ ]:
```
# import fizzbuzz

from fizzbuzz import fizzbuzz
import pytest

# check the function returns numeric inputs as string

def test_return_as_string():
    assert fizzbuzz(1) == "1"

#test function works for multiples of 3

def test_multi_three():
    assert fizzbuzz(3) == "Fizz"


# test function works for multiples of 5

def test_multi_five():
```

```
    assert fizzbuzz(5) == "Buzz"


# test function works for multiples of 15

def test_multi_fifteen():
    assert fizzbuzz(15) == "Fizzbuzz"

# start testing for errors

#check that input is int
def test_non_int_input():
    with pytest.raises(TypeError):
        fizzbuzz("hello!")


def test_non_negative_input():
    with pytest.raises(ValueError):
        fizzbuzz(-1)
```

GitHub Actions Workflow (YML)

Below is the code for the .yml file used for the GitHub actions workflow for testing the function.

```
name: Fizzbuzz tests

# specify when we want this to be triggered
# we want to trigger it for every push and PR

on: [push, pull_request]

#specify the jobs needed for the CI

jobs:
  tests:
    runs-on: ubuntu-latest

    steps:

    - uses: actions/checkout@v4

    - name: Set up python
      uses: actions/setup-python@v5
      with:
        python-version: "3.x" # to use any version of python 3

#make sure we have pytest and upgrades if needed
    - name: Install dependencies
      run: pip install --upgrade pip pytest
# run the actual test
    - name: Run pytest
      run: pytest
```